

Arbitrary Precision Arithmetic Library

Approach and Design Report

cs24btech11038

B.Tech, Year 1, IIT Hyderabad

May 2, 2025

1 Approach

The main idea behind this project was to build a library that can handle very large numbers (both integers and decimals) accurately, without worrying about the limits of Java's regular `int` or `double` types. In real-world scenarios like banking or scientific computations, precision matters a lot, and regular data types just aren't enough. That's why this project uses Java's `BigInteger` and `BigDecimal` classes.

My approach was to divide the project into two main parts:

- A core Java library that provides classes for big integers and floats.
- A command-line interface (CLI) where users can input numbers and operations to see results.

I first created an interface named `Arithmetic<T>` that defines four basic operations: `add`, `subtract`, `multiply`, and `divide`. Then I made two classes—`AInteger` and `AFloat`—which implement this interface. These classes wrap around `BigInteger` and `BigDecimal` respectively.

2 Design

The design is based on object-oriented programming, which helps keep things clean and modular. Here's a breakdown:

`AInteger` and `AFloat`

These two classes handle large integers and floating-point numbers. They provide the four basic arithmetic functions by using Java's built-in precision libraries. Each class has a constructor that takes a string and methods like `add()`, `subtract()`, etc., which return new objects instead of changing the original one.

Interface for Reusability

The `Arithmetic<T>` interface was created so that any type of number—be it big integers, floats, or even complex numbers in the future—can follow the same structure. This makes the code more reusable and easier to extend later on.

Command Line Tool

I created a class called `MyInfArith` that acts as the entry point. It reads the user's input, detects whether it's a float or an integer, and then calls the appropriate class and method to perform the operation. This makes the tool user-friendly and good for testing.

Error Handling

Proper error handling is included to manage cases like:

- Division by zero
- Invalid inputs (e.g., entering alphabets)

This keeps the program from crashing and gives helpful feedback to the user.

Why This Design Works

This design separates the logic clearly:

- The interface defines the rules
- The classes implement them
- The CLI connects everything together

It's also easy to add more features in the future, like exponentiation, square roots, or support for negative formatting and rational numbers.

3 Conclusion

This project helped me understand how to handle precision issues in Java, and how to design clean, reusable code using interfaces and classes. The CLI makes it easy to interact with the library, and the modular design allows for future growth.